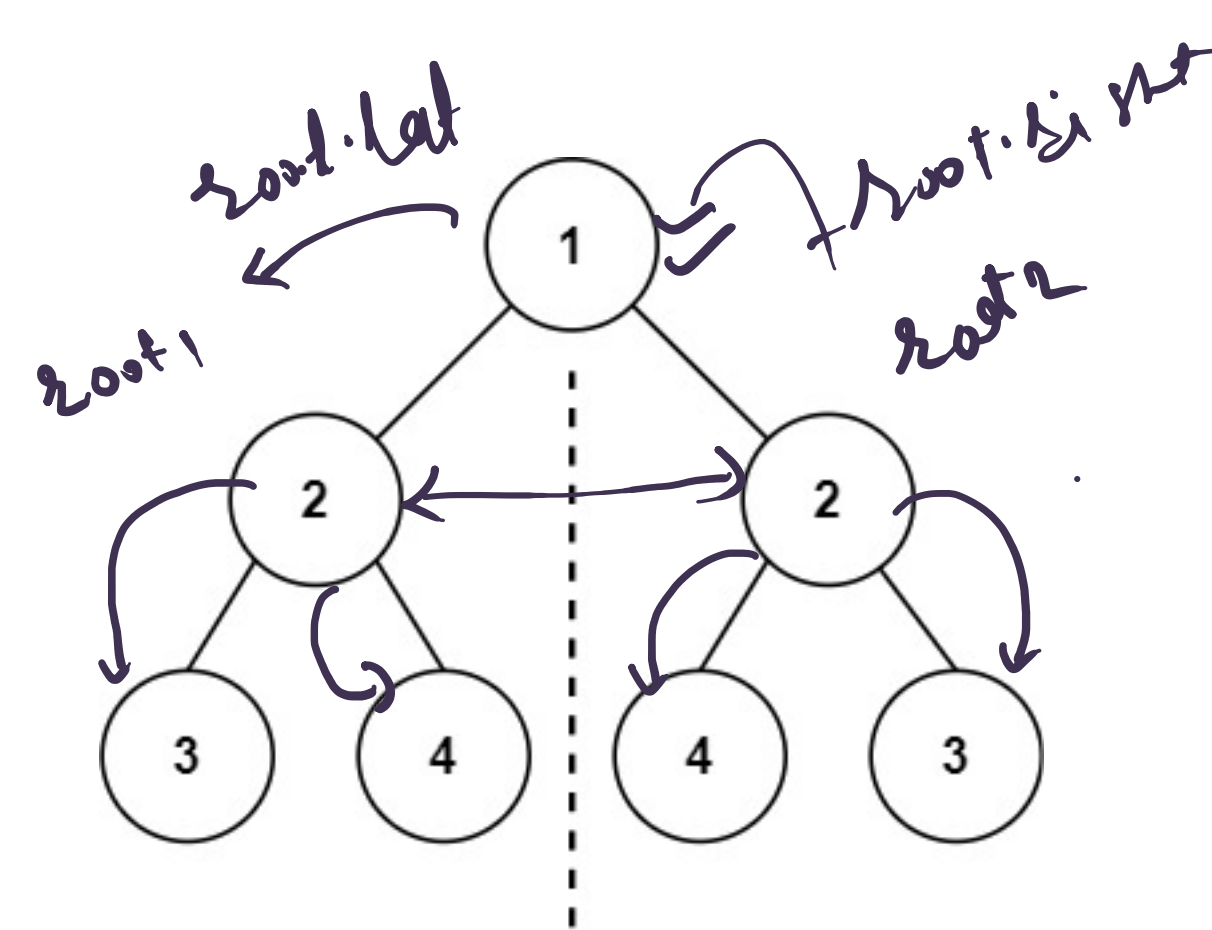
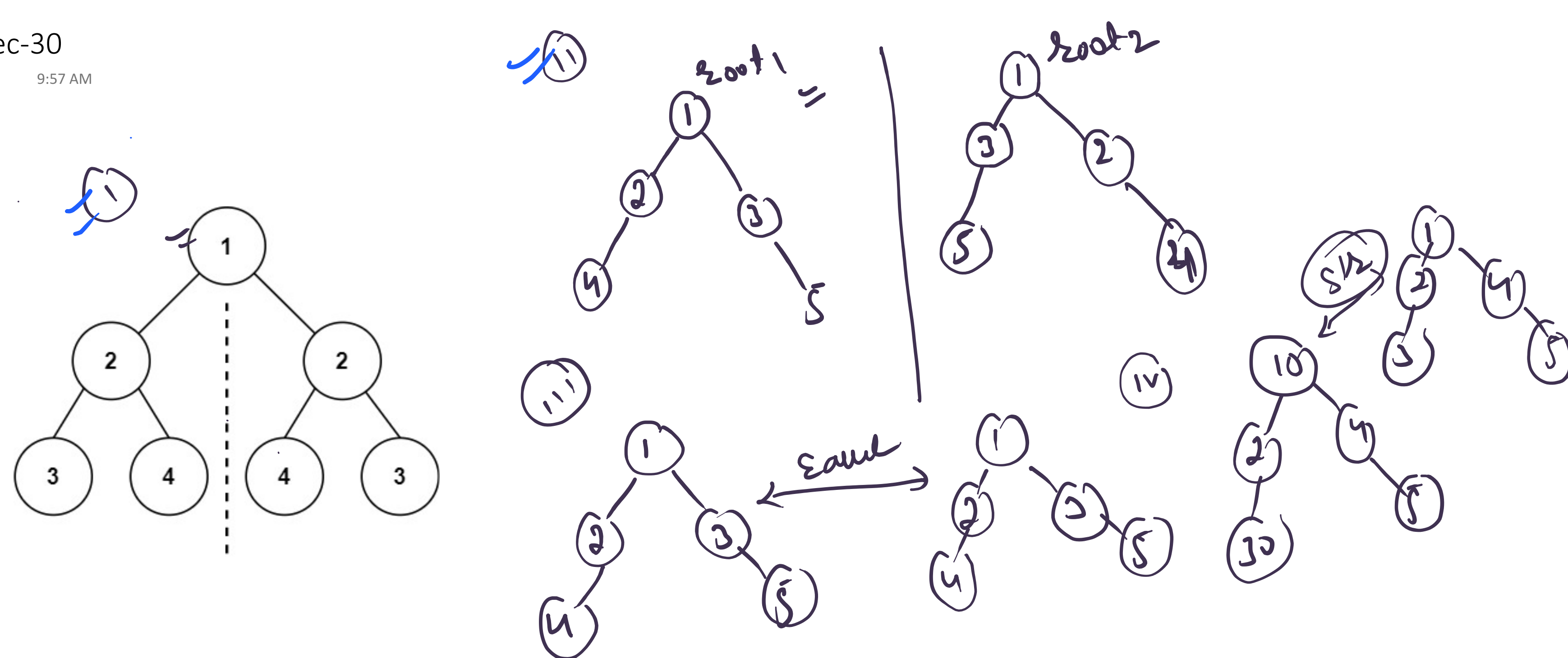




① $z_{0011} = \text{null}$ & $z_{0012} = \text{null}$
true

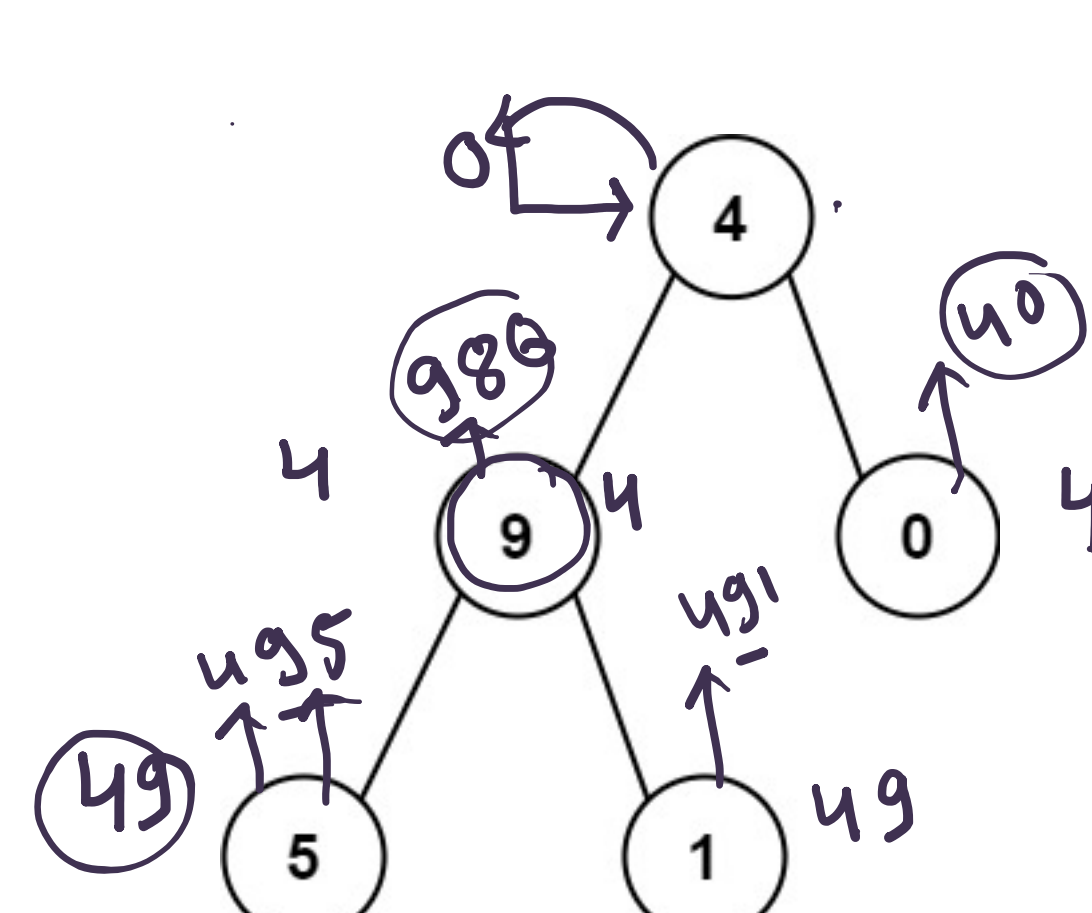
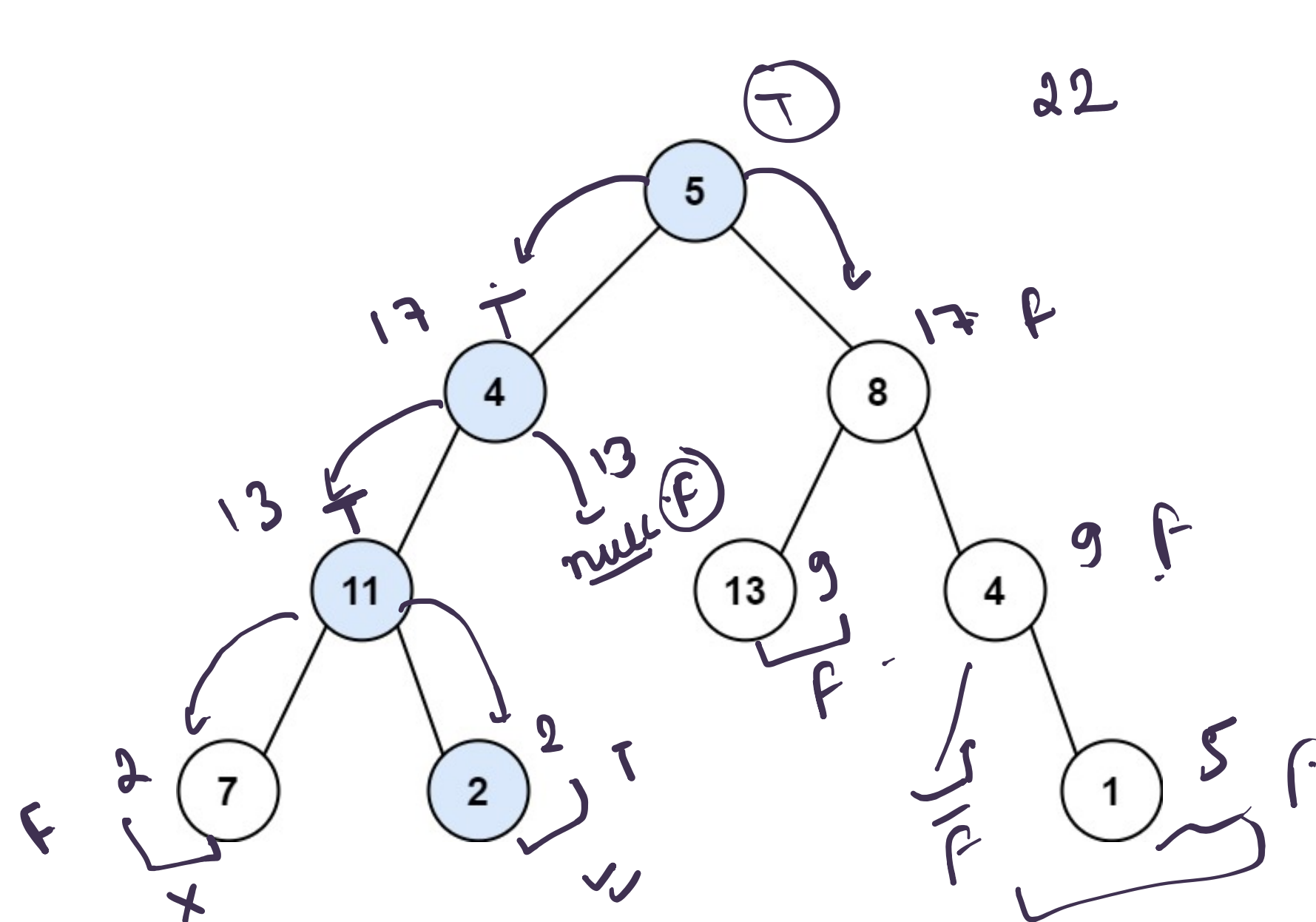
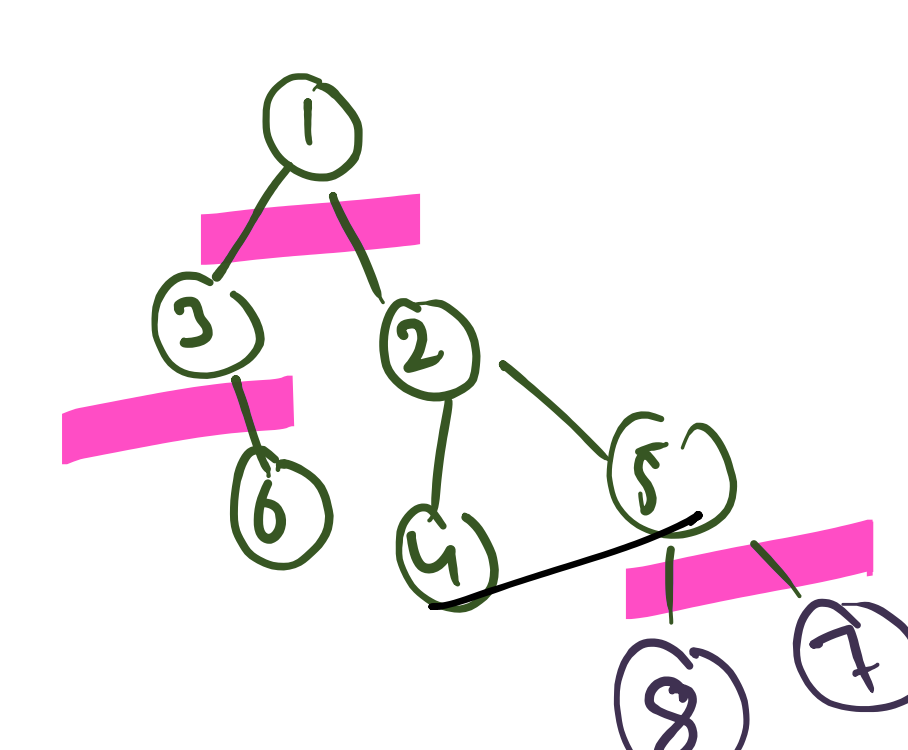
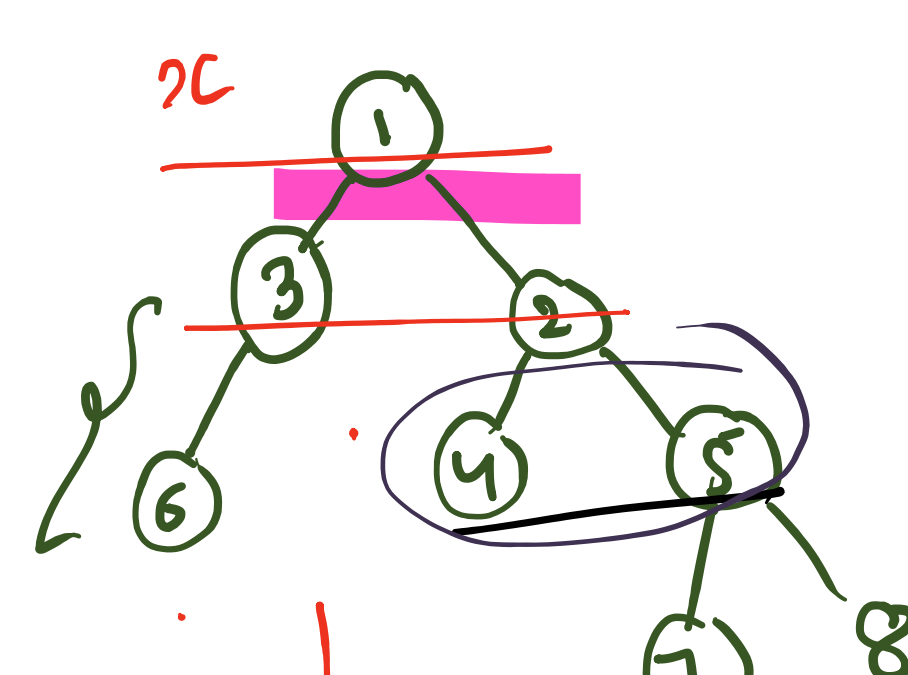
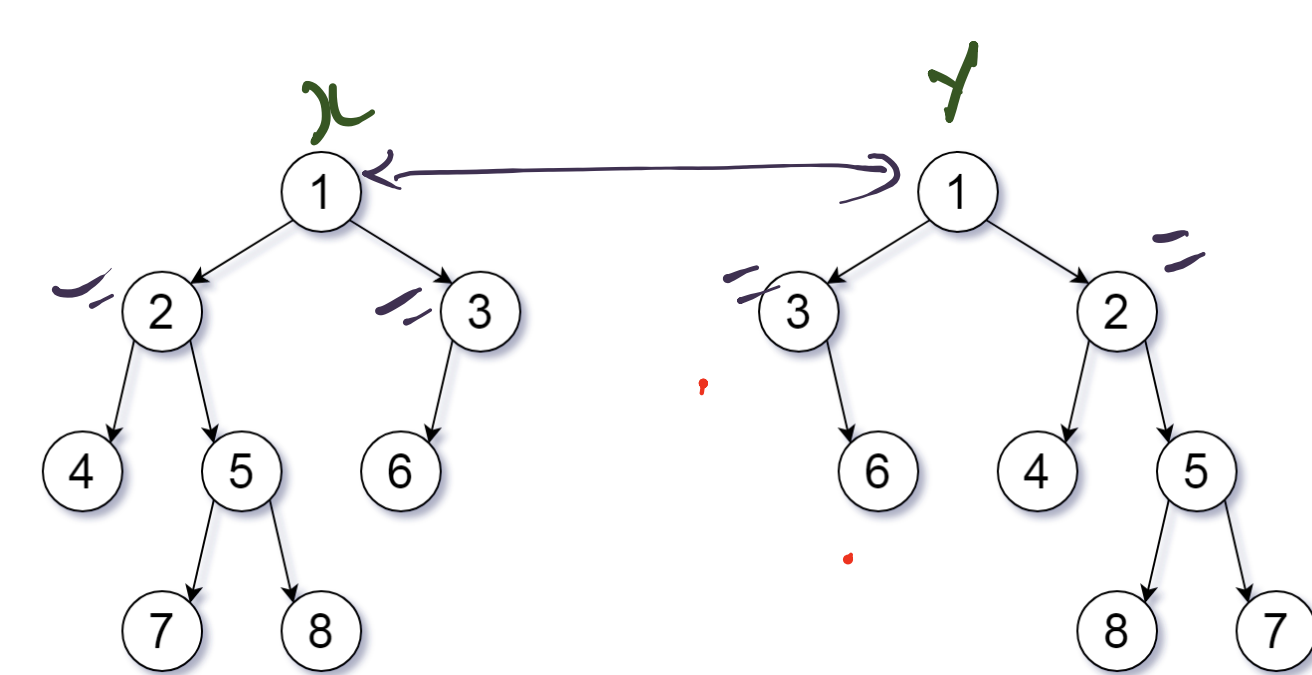
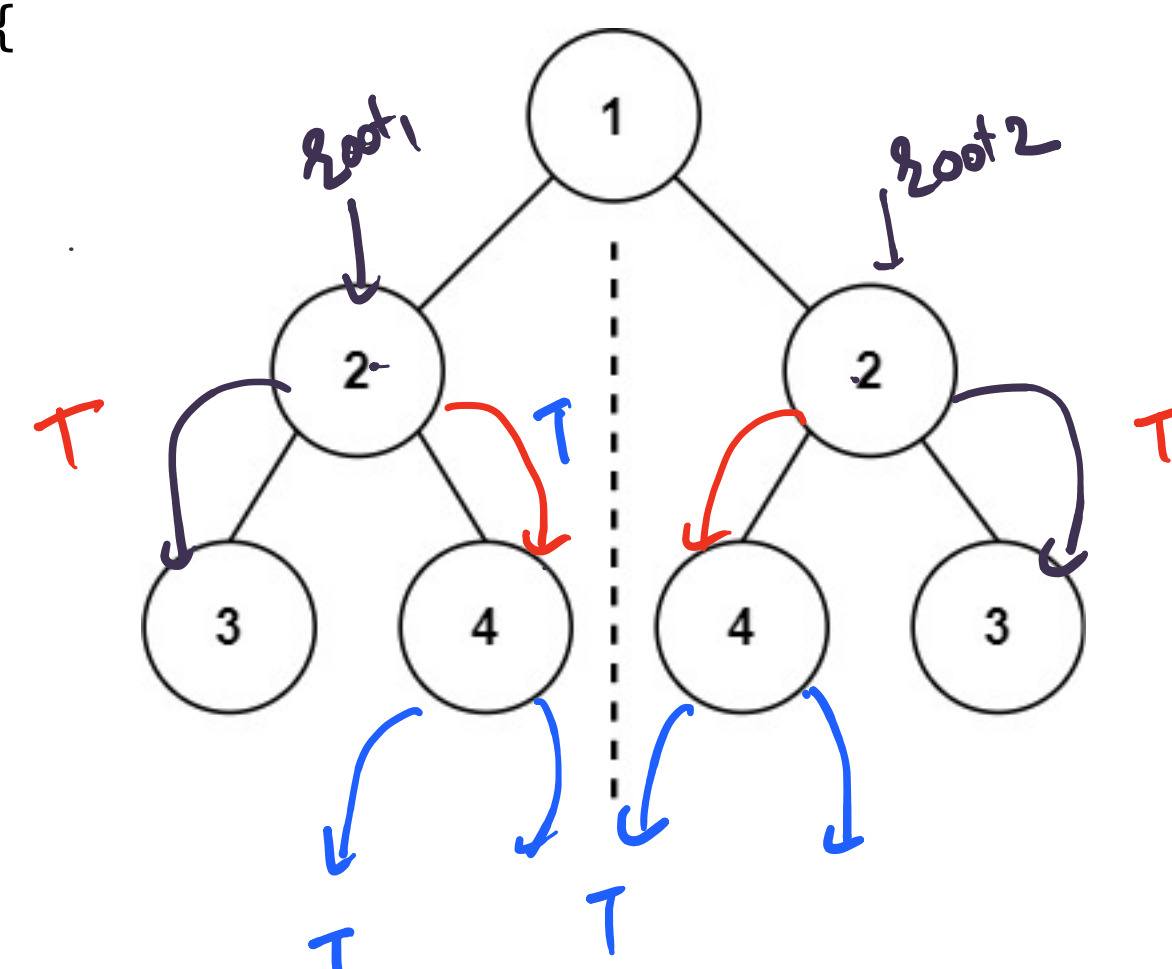
② $z_{0011} = \text{null}$ || $z_{0012} = \text{null}$
false
 $z_{0011} \cdot \text{val} != z_{0012} \cdot \text{val}$
even

```
public boolean isSymmetric(TreeNode root) {
    return IsMirror_Image(root.left, root.right);
}
```

```

public boolean IsMirror_Image(TreeNode root1, TreeNode root2) {
    if (root1 == null && root2 == null) {
        return true;
    }
    if (root1 == null || root2 == null) {
        return false;
    }
    if (root1.val != root2.val) {
        return false;
    }
    boolean left = IsMirror_Image(root1.left, root2.right);
    boolean right = IsMirror_Image(root1.right, root2.left);
    return left && right;
}

```



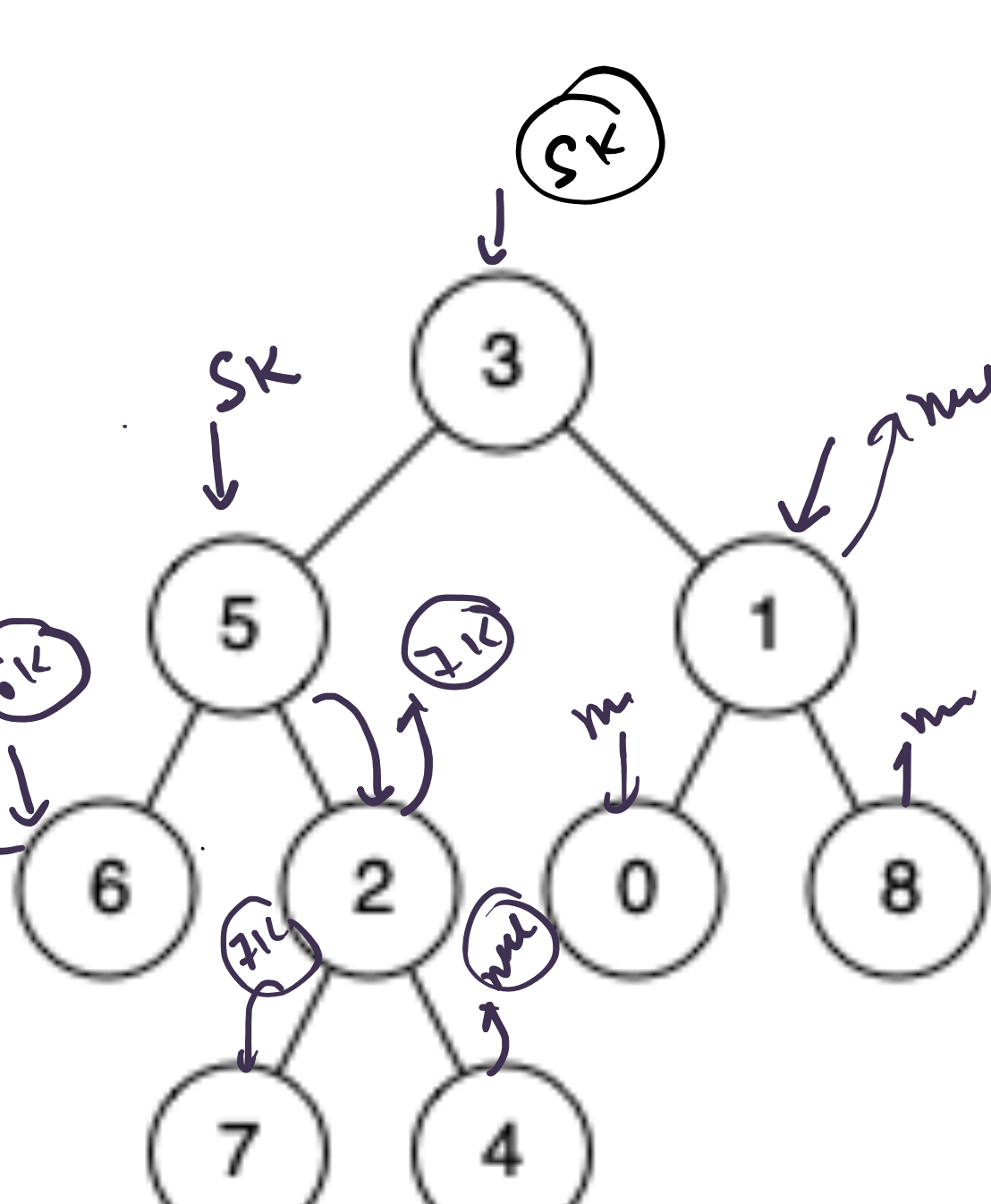
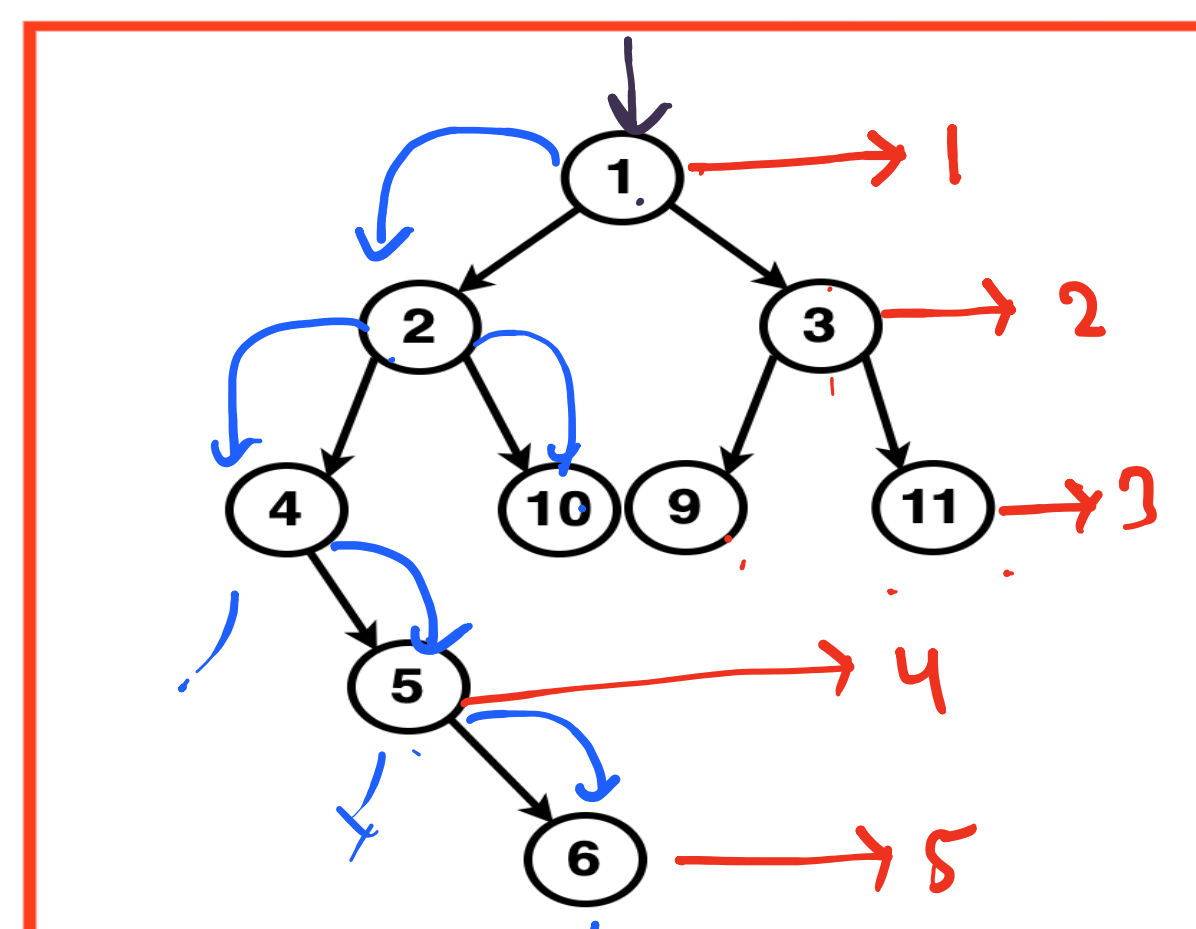
1026

nom x 10 + 2001. val

max depth = 0 1 2 3 4 5

```
if (currLen > maxLen)
```

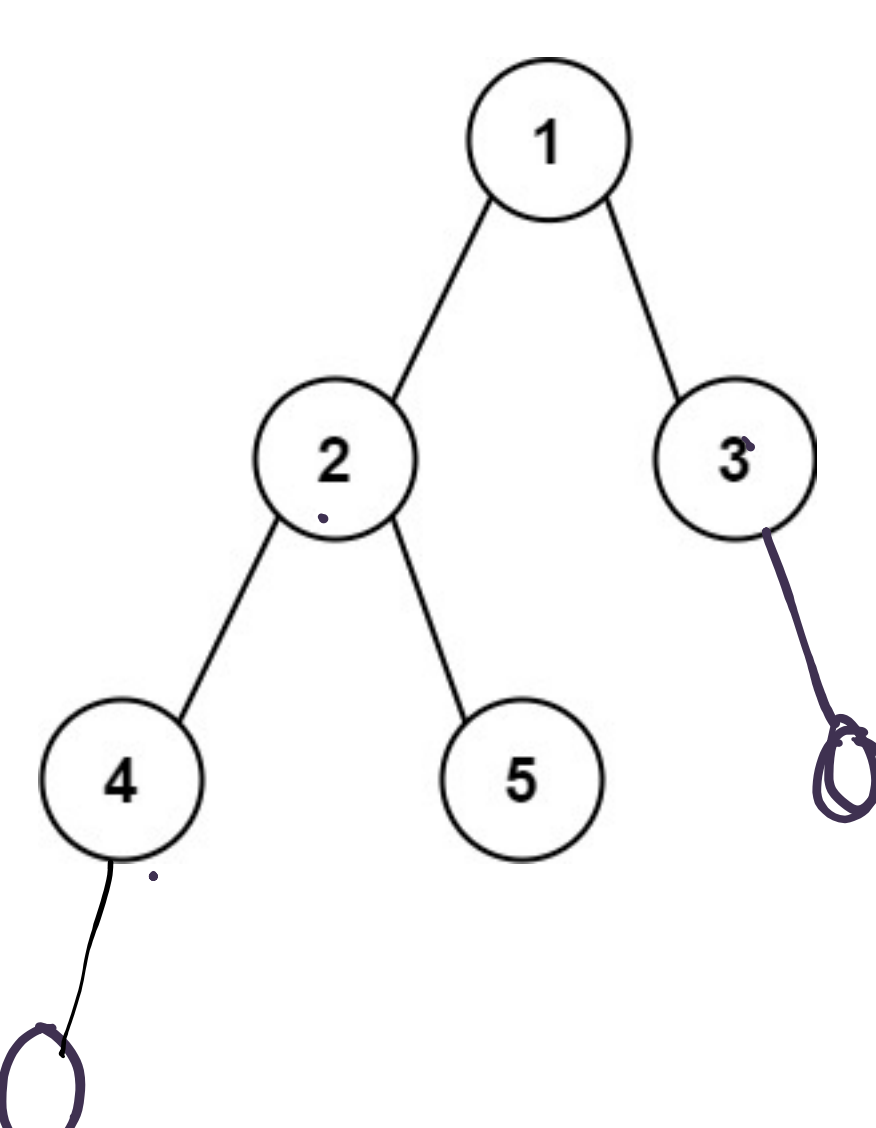
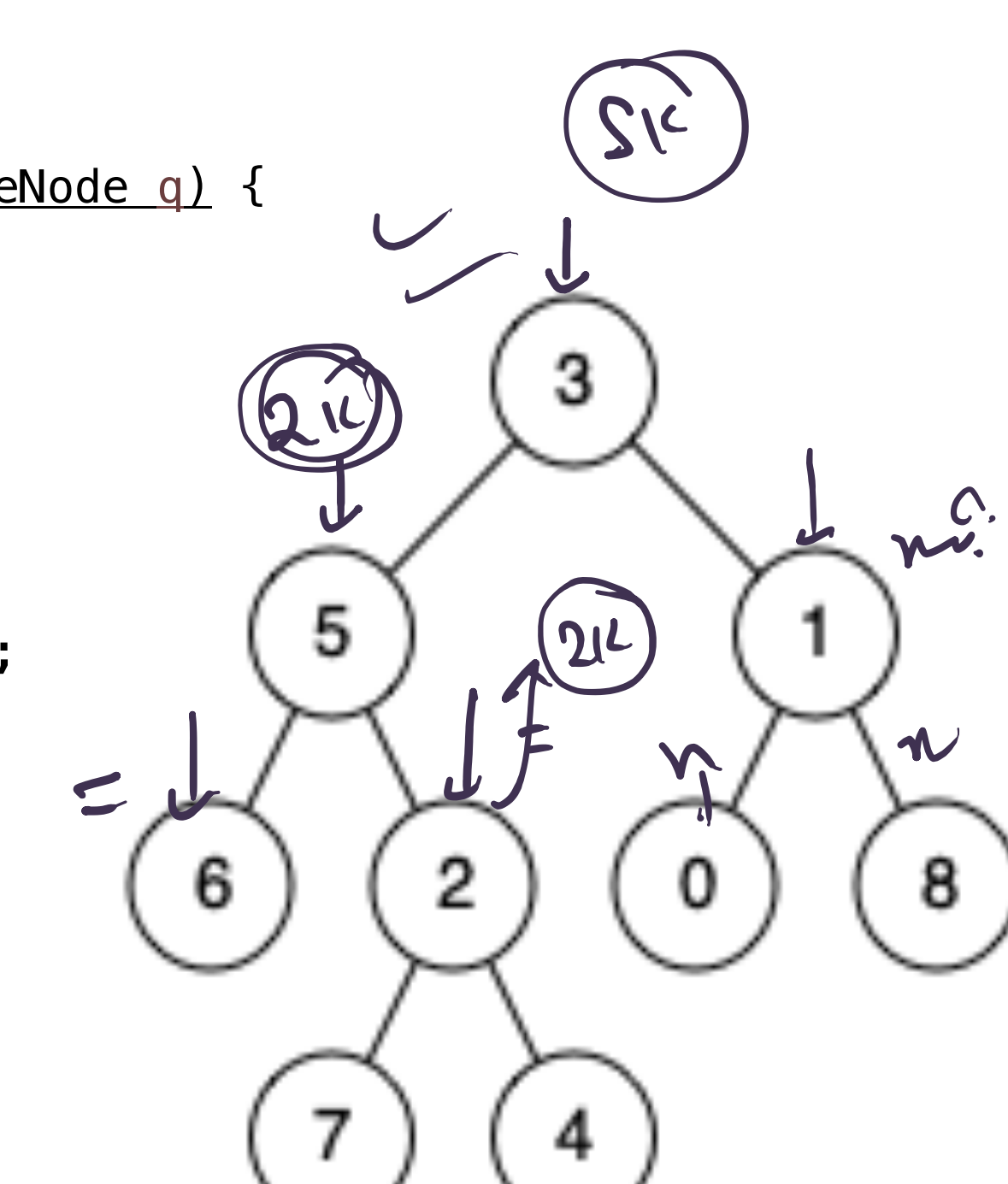
2 11 5 7 2



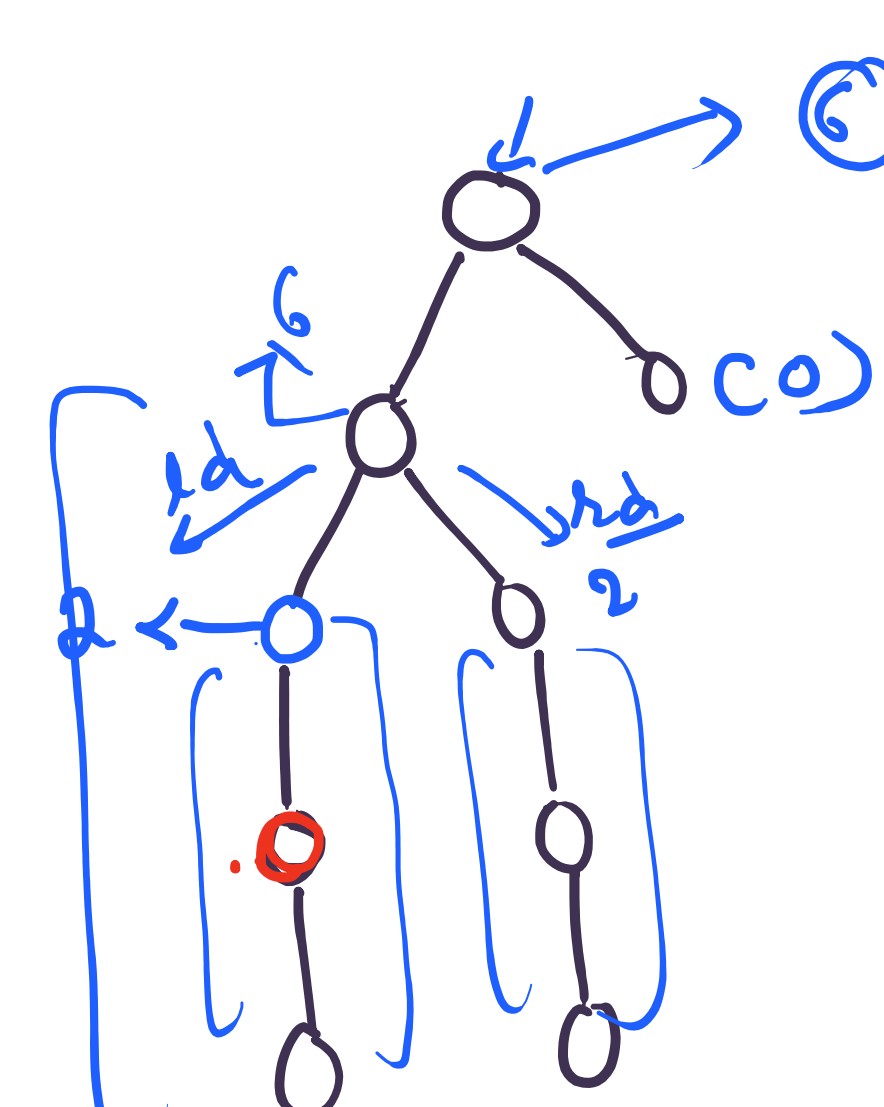
6, 7

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) {
        return null;
    }
    if (root == p || root == q) {
        return root;
    }
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) {
        return root;
    }
    else if (left == null) {
        return right;
    }
    else {
        return left;
    }
}
```


$$lh + Rh + 2$$
$$3 + 0 + 2 = 5$$
$$2 + 2 + 2 = 6$$

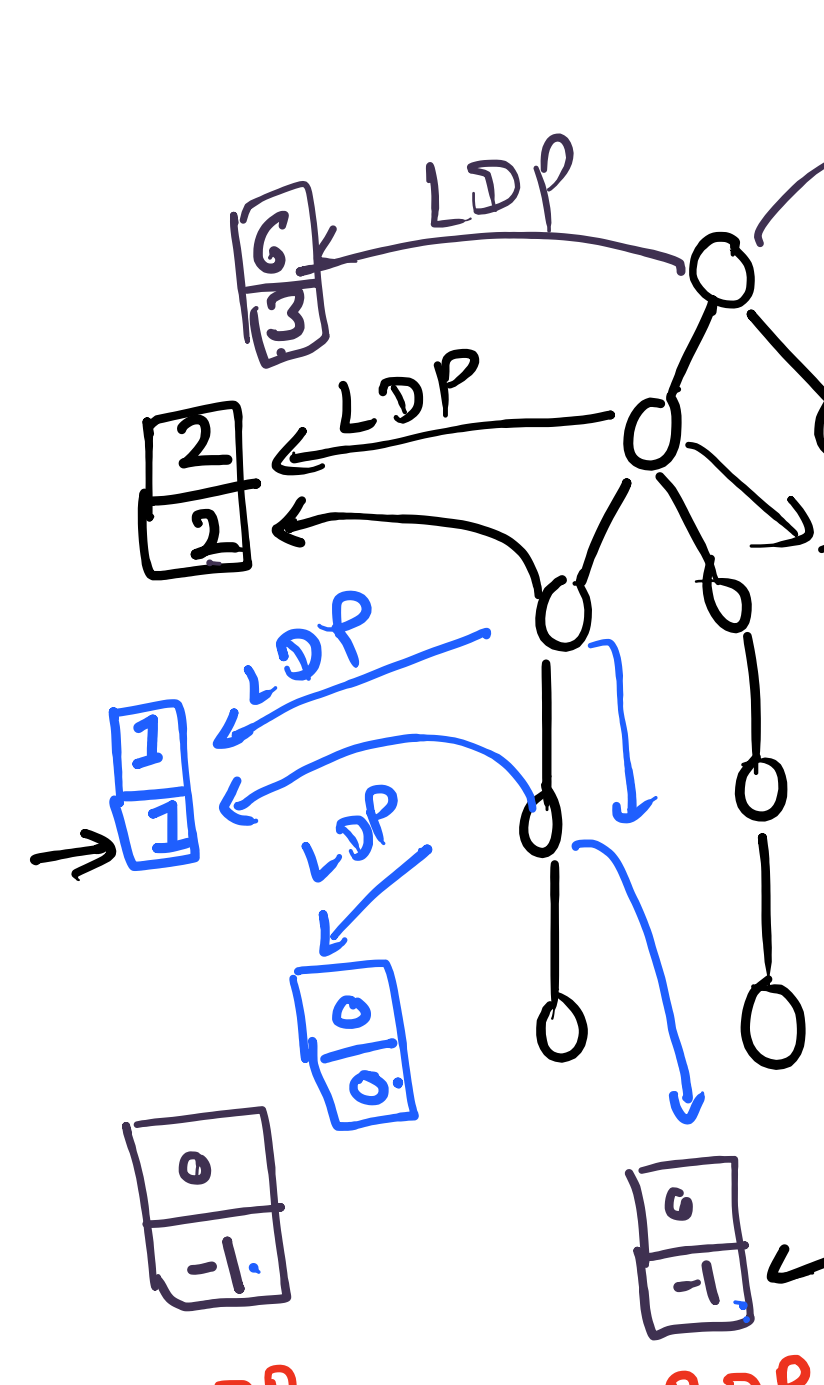
```
public int diameterOfBinaryTree(TreeNode root) {
    if (root == null) {
        return 0;
    }
    int ld = diameterOfBinaryTree(root.left);
    int rd = diameterOfBinaryTree(root.right);
    int sd = ht(root.left) + ht(root.right) + 2;
    return Math.max(ld, Math.max(rd, sd));
}
```


$$1 - 1 + 2 = 2$$

```

public DiaPair diameter(TreeNode root) {
    if (root == null) {
        return new DiaPair();
    }
    DiaPair ldp = diameter(root, left);
    DiaPair rdp = diameter(root, right);
    int sd = ldp.ht + rdp.ht + 2;
    DiaPair sdp = new DiaPair();
    sdp.dia = Math.max(ldp.dia, Math.max(rdp.dia, sd));
    sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
    return sdp;
}

```


$$-1 - 1 + 2 = 0$$

LAP

RDF